

Εργασία Τεχνητής Νοημοσύνης

Γεωργιάδης Γεώργιος, AM: 4038

Βασίλειος Καλτσούνης, AM: 4600

05/05/2026

Άσκηση 1

Το πρόβλημα που κληθήκαμε να λύσουμε αφορά το γνωστό N-puzzle και συγκεκριμένα την περίπτωση του 8-puzzle, δηλαδή ένα 3x3 πλέγμα με 8 πλακάκια με αριθμούς από 1 έως 8 και μια κενή θέση. Σκοπός είναι μετακινώντας τα πλακίδια στη κενή θέση να τα αναδιοργανώσουμε έτσι ώστε να φτάσουμε στη κατάσταση-στόχο (τελική κατάσταση):

1 2 3		0 1 2
6 5 4	με αρίθμηση θέσεων:	3 4 5
7 0 8		6 7 8

Οι κινήσεις που επιτρέπονται είναι οι εξής:

UP, DOWN, LEFT, RIGHT με κόστος 1.0

WRAP LEFT, WRAP RIGHT, WRAP UP, WRAP DOWN (δηλαδή μεταφορά του πλακιδίου από τη 1η στήλη στη 3η και αντίστροφα στην ίδια γραμμή ή μεταφορά του πλακιδίου από τη 1η γραμμή στη 3η και αντίστροφα στην ίδια στήλη) με κόστος 1.0

TELEPORT (δηλαδή η μεταφορά του πλακιδίου από τη θέση 2 στη θέση 6 και αντίστροφα) με κόστος 0.5.

Αρχικά εφαρμόσαμε αναζήτηση ομοιόμορφου κόστους (UCS) για να μπορέσουμε να έχουμε μία πρώτη εκτίμηση για τη λύση του προβλήματος και έπειτα υλοποιήσαμε την ευρετική μας συνάρτηση για να έχουμε και προσέγγιση του προβλήματος με A* και να συγκρίνουμε τις επεκτάσεις που χρειάστηκαν.

Η αρχική μας προσέγγιση για την επιλογή ευρετικής συνάρτησης ήταν η απόσταση Manhattan. Παρ'όλα αυτά η απόσταση Manhattan, εφόσον υπολογίζει μόνο κάθετες και οριζόντιες κινήσεις σε ένα πλέγμα, θα οδηγούσε σε μη αποδεκτή ευρετική καθώς το κόστος που θα υπολόγιζε για ορισμένες κινήσεις θα ήταν μεγαλύτερο από το πραγματικό. Γι'αυτόν τον λόγο αποφασίσαμε να τη τροποποιήσουμε έτσι ώστε να λαμβάνει υπόψη τις έξτρα κινήσεις.

```

31 // epistrefei to elaxisto kostos kinhsewn pou apaiteitai gia na metakinithei ena plakidio
32 // apo th thesh a sth thesh b, lambanontas upopshn treis diadromes: apeutheias (me kinhseis up/down/le
33 // me tis klassikes kinhseis + teleport apo 2 se 6 kai me klassikes kinhseis + teleport apo 6 se 2
34 private static double distance(int a, int b) {
35     double direct = manhattan(a, b);
36
37     double tpDown = manhattan(a, 2) + 0.5 + manhattan(6, b);
38     double tpUp = manhattan(a, 6) + 0.5 + manhattan(2, b);
39
40     if (tpDown < direct) {
41         direct = tpDown;
42     }
43
44     if (tpUp < direct) {
45         direct = tpUp;
46     }
47
48     return direct;
49 }
50
51 // tropopoihmenh apostash manhattan gia na lambanei upopshn tis wrap kinhseis
52 // allazei thn apostash gia tis wrap kinhseis apo 2 se 1 giati metakineitai sthn ousia duo theseis (a
53 // alla kostizei 1.0 san tis kanonikes kinhseis
54 private static double manhattan(int a, int b) {
55     int curRow = a / 3;
56     int curCol = a % 3;
57
58     int goalRow = b / 3;
59     int goalCol = b % 3;
60
61     int rowDiff = curRow - goalRow;
62     int colDiff = curCol - goalCol;
63
64     if (rowDiff < 0) {
65         rowDiff = -rowDiff;
66     }
67     if (colDiff < 0) {
68         colDiff = -colDiff;
69     }
70     if (rowDiff == 2) {
71         rowDiff = 1;
72     }
73     if (colDiff == 2) {
74         colDiff = 1;
75     }
76     return rowDiff + colDiff;
77 }
78 }

```

Στις γραμμές 37-38 το κόστος ενός μονοπατιού a-b, όπου a η τωρινή θέση του πλακιδίου και b η θέση στόχος (στο goal state), υπολογίζεται από την απόσταση της θέσης a από την θέση 2, το κόστος του teleport, και την απόσταση της θέσης 6 από τη θέση b. Το ίδιο συμβαίνει και για την ανάποδη διαδρομή.

Οι διαδρομές χωρίς teleport υπολογίζονται από απλή εφαρμογή της manhattan στα a και b και αποθηκεύονται σε μια μεταβλητή direct. Αν το κόστος κάποιου από τα δύο μονοπάτια με teleport (tpUp και tpDown) είναι μικρότερο από το direct που υπολογίζει η Manhattan, ο αλγόριθμος επιστρέφει αυτά στη θέση του direct. Επιπλέον, στις γραμμές 70-74, προκειμένου οι κινήσεις wrap να έχουν σωστό κόστος, αν η διαφορά μεταξύ των γραμμών (η αντίστοιχα των στηλών) είναι 2, αυτό σημαίνει ότι είναι σε ακριανές γραμμές/στήλες και μετατρέπεται το κόστος σε 1. Με αυτόν τον τρόπο, η A* θα πρέπει να ψάξει ένα μονοπάτι με αυτό το μειωμένο κόστος και έτσι θα οδηγηθεί στην επέκταση που αφορά την εκάστοτε κίνηση wrap.

Κατασκευάσαμε την ευρετική συνάρτηση βασισμένοι στο γεγονός ότι η απόσταση Manhattan είναι η λύση ενός χαλαρωμένου προβλήματος στο οποίο τα πλακίδια μπορούν να μετακινηθούν σε οποιαδήποτε θέση (στη περίπτωση μας στη θέση στόχο) ανεξάρτητα από τη θέση των υπολοίπων πλακιδίων. Κατά αυτόν τον τρόπο ο υπολογισμός αυτής της

τροποποιημένης απόστασης Manhattan δημιουργεί ένα κάτω όριο για το κόστος που χρειάζεται το πλακίδιο για να μετακινηθεί στη θέση στόχο. Έτσι η A* δε θα λάβει υπερεκτιμημένο κόστος για κάποια κίνηση, καθώς το ελάχιστο κόστος που λαμβάνει είναι το άθροισμα των ελαχίστων τιμών κόστους το οποίο δεν μπορεί να γίνει μικρότερο. Συνεπώς η ευρετική συνάρτηση είναι αποδεκτή.

Επεξηγήσεις κώδικα

State.java

```
1  import java.util.Arrays;
2
3  // antiprosopoeiei enan pinaka puzzle 3x3 ws ena grammiko array 9 grammwn
4  // me index 0..8 gia eukoloterh diaxeirhsh
5  public class State {
6      int[] board; // ta plakidia tou game, 0 shmainei keno
7
8      // h telikh katastash pou theloume na ftasei
9      public static final int[] goal = {1, 2, 3, 6, 5, 4, 7, 0, 8};
10
11     public State(int[] board) {
12         this.board = Arrays.copyOf(board, 9);
13     }
14
15     public boolean goal() {
16         for (int i = 0; i < 9; i++) {
17             if (board[i] != goal[i]) {
18                 return false;
19             }
20         }
21
22         return true;
23     }
```

Η κλάση State διατηρεί έναν μονοδιάστατο πίνακα ακεραίων board ο οποίος αναπαριστά την εκάστοτε κατάσταση του προβλήματος, καθώς το index του κάθε στοιχείου του array αντιστοιχεί σε μία θέση στο 3x3 board (0..8) και η κάθε τιμή που εμπεριέχει αντιστοιχεί στη τιμή του πλακιδίου του board στο συγκεκριμένο index.

Επίσης αποθηκεύει στην ίδια μορφή την αποδεκτή τελική κατάσταση (goal) και μας προμηθεύει με συναρτήσεις απαραίτητες για την διαχείριση του προβλήματος.

Η συνάρτηση goal επιστρέφει true εφόσον το board και το goal τα οποία είναι πεδία της State ταυτίζονται.

```
24
25     // epistrefei to index ths theshs tou pinaka pou einai keno, h -1 an den brethei (sfalma)
26     public int findBlank() {
27         for (int i = 0; i < 9; i++) {
28             if (board[i] == 0) {
29                 return i;
30             }
31         }
32         return -1;
33     }
34
35     // kwdikopoei ton pinaka ws enan akeraio 9 psifiwn gia eukoloterh diaxeirhsh (hashing)
36     public int hashCode() {
37         int hash = 0;
38
39         for (int i = 0; i < 9; i++) {
40             hash = hash * 10 + board[i];
41         }
42
43         return hash;
44     }
```

Η findBlank() είναι κρίσιμη συνάρτηση καθώς μας επιστρέφει το index της θέσης του κενού, το οποίο είναι απαραίτητο για να προκύψουν οι επεκτάσεις. Η hashCode() ουσιαστικά

μετατρέπει τον πίνακα 3x3 σε έναν ακέραιο αριθμό 9 ψηφίων έτσι ώστε να μας επιτραπεί ευκολότερη διαχείριση και εκμετάλλευση.

Node.java

```
1 // h klash pou dhmiourgei to antikeimeno tupou kombou
2 public class Node {
3     State state; // h katastash tou kombou
4     Node prevNode; // o kombos proeleushs (goneas)
5     String move; // h kinshh apo thn opoia proekupse autos o kombos
6     double g; // to pragmatiko kostos apo thn AK sth twrinh katastash-kombo
7     double h; // h ektimhsh ths euretikhs sunarthshs gia to upoloipomeno kostos (0 gia UCS, upologiz
8
9     public Node(State state, Node prevNode, String move, double g, double h) {
10         this.state = state;
11         this.prevNode = prevNode;
12         this.move = move;
13         this.g = g;
14         this.h = h;
15     }
16
17     // epistrefei to sunoliko ektimoumeno kostos f = g + h to opoio xrhsimopoiiei h A*
18     // prokeimenou na organwsei th seira proteraiothtas (priority queue)
19     public double f() {
20         return g + h;
21     }
22 }
23
```

Η κλάση Node δημιουργεί το αντικείμενο Node το οποίο διατηρεί όλες τις απαραίτητες πληροφορίες των θεωρούμενων κόμβων του γράφου επεκτάσεων. Επίσης, εφόσον σε αυτό αποθηκεύονται η απόσταση (κόστος) $g(n)$ του κόμβου από την ΑΚ και η τιμή της ευρετικής συνάρτησης ($h(n)$), η κλάση παρέχει μια μέθοδο για την εύρεση του συνολικού εκτιμώμενου κόστους ($f = g + h$) και το επιστρέφει.

Moves.java

```
1 import java.util.ArrayList;
2
3 // paragei oles tis egkures kinhseis apo enan kombo
4 public class Moves {
5
6     // epistrefei lista me ola ta paidia tou trexontos kombou
7     public static ArrayList<Node> getMoves(Node node) {
8         ArrayList<Node> move = new ArrayList<>();
9
10        State state = node.state;
11
12        // briksai thn thesh tou kenou kai upologizei th grammh kai th sthlh tou
13        int blank = state.findBlank();
14
15        int row = blank / 3;
16        int col = blank % 3;
17
18        // kanonikes kinhseis (panw, katw, deksia, aristera) me kostos 1.0
19        if (row > 0) {
20            move.add(makeNode(node, blank, blank - 3, "UP", 1.0));
21        }
22        if (row < 2) {
23            move.add(makeNode(node, blank, blank + 3, "DOWN", 1.0));
24        }
25        if (col > 0) {
26            move.add(makeNode(node, blank, blank - 1, "LEFT", 1.0));
27        }
28        if (col < 2) {
29            move.add(makeNode(node, blank, blank + 1, "RIGHT", 1.0));
30        }
31    }
32 }
```

Η κλάση moves ορίζει ξεκάθαρα όλες τις πιθανές κινήσεις μαζί με το κόστος τους. Η μέθοδος getMoves(node) επιστρέφει ένα ArrayList τύπου move με όλες τις πιθανές κινήσεις που

μπορούν να πραγματοποιηθούν από τον κόμβο node (όρισμα) προκειμένου να γίνουν οι πιθανές επεκτάσεις.

```
58
59 // dhmiourgei enan neo kombo emfarmozontas th kinsh (antallagh oustiasatika kenou me plakidio)
60 private static Node makeNode(Node prevNode, int blank, int tile, String moves, double cost) {
61     int[] newBoard = new int[9];
62
63     for (int i = 0; i < 9; i++) {
64         newBoard[i] = prevNode.state.board[i];
65     }
66     newBoard[blank] = newBoard[tile];
67     newBoard[tile] = 0;
68     State newState = new State(newBoard);
69     return new Node(newState, prevNode, moves, prevNode.g + cost, 0);
70 }
71 }
```

Τέλος, η μέθοδος makeNode δημιουργεί έναν πανομοιότυπο πίνακα (στιγμιότυπο) με αυτόν του προηγούμενου κόμβου, δημιουργεί μια νέα κατάσταση για τον νέο κόμβο που προκύπτει αφού εφαρμόσει την κίνηση που προκαλεί την επέκταση (δηλαδή την εναλλαγή του κενού με κάποιον γείτονα του) και τελικά επιστρέφει τον νέο κόμβο.

UCS.java

```
5 // atropiei ton algorithmo uniform cost search
6 // xrhsimopoei priority queue me proteraiothta to kostos diadromhs (g)
7 public class UCS {
8
9     public static Node search(State start) {
10         // priority queue taksinomhmenh kata auksousa timh g
11         // hash set gia na mh ksanapiskefthoume thn idia katastash
12         PriorityQueue<Node> nodes = new PriorityQueue<>((a, b) -> Double.compare(a.g, b.g));
13         HashSet<State> used = new HashSet<>();
14
15         Node node = new Node(start, null, null, 0, 0);
16         nodes.add(node);
17
18         int expansions = 0;
19
20         while (nodes.isEmpty() == false) {
21             Node cur = nodes.poll();
22
23             // an h katastash exei epektathe
24             // agnooume
25             if (used.contains(cur.state)) {
26                 continue;
27             }
28
29             used.add(cur.state);
30
31             // an ftasoume ston stocho (tk), epistrefoume ton kombo
32             if (cur.state.goal()) {
33                 System.out.println("expansions :" + expansions);
34                 return cur;
35             }
36
37             expansions++;
38
39             List<Node> moves = Moves.getMoves(cur);
40
41             // paragwgh kai prosthiki twn apodektnw kinhsewn sto metwpo
42             for (Node move : moves) {
43                 if (used.contains(move.state) == false) {
44                     nodes.add(move);
45                 }
46             }
47         }
48
49         System.out.println("expansions :" + expansions);
50         return null;
51     }
52 }
```

Η κλάση UCS υλοποιεί την αναζήτηση ομοιόμορφου κόστους. Επιλέξαμε να χρησιμοποιήσουμε PriorityQueue για να μπορούμε να έχουμε τους κόμβους που δημιουργούνται σε κάθε βήμα ταξινομημένους με βάση την απόσταση από την ΑΚ ($g(n)$), έτσι ώστε η κεφαλή της ουράς, η οποία θα είναι αυτή με το μικρότερο κόστος $g(n)$, να επιστραφεί από την κλήση poll() που δημιουργεί τον νέο κόμβο που προέκυψε από την επέκταση.

Επιπλέον, χρησιμοποιούμε ένα HashSet για να σημειώσουμε τους ήδη επεκταμένους κόμβους, καθώς η δομή αυτή δεν επιτρέπει διπλότυπα με μεγάλη αυστηρότητα, οπότε δε θα χρειαστεί να ξαναεπισκεφθεί κάποιον κόμβο και έτσι αποφεύγουμε τρομερή πολυπλοκότητα. Ειδικά, οι wrap κινήσεις θα μπορούσαν να δημιουργούν ατέρμονους κύκλους και το πρόγραμμα να μη τερματίζει ποτέ, αλλά με τη HashSet used αυτό αποκλείεται να συμβεί.

Astar.java

```
37
38     List<Node> moves = Moves.getMoves(current);
39
40     // paragwgh kai prosthiki twn apodektnw kinhsewn sto metwpo me upologismo kostous kinshs
41     for (Node move : moves) {
42         if (used.contains(move.state) == false) {
43             move.h = Heuristic.compute(move.state);
44             nodes.add(move);
45         }
46     }
47 }
48
49 System.out.println("expansions : " + expansions);
50 return null;
51 }
52 }
53 }
```

Στη κλάση Astar λειτουργούμε αναλόγως με τη κλάση UCS όσον αφορά τη δομή του προγράμματος, με τη μόνη διαφορά ότι υπολογίζουμε μέσω της μεθόδου compute της κλάσης Heuristic τη τιμή της ευρετικής συνάρτησης ($h(n)$) για να τη προσθέσουμε στο εκάστοτε node.

Πειραματικά Αποτελέσματα

```
giorgosgeorgiades@Giorgoss-MacBook-Air askisi1_2 % java Main
Enter 1 row of numbers (3 numbers, 0 = blank), space-separated:
1 0 3
Enter 2 row of numbers (3 numbers, 0 = blank), space-separated:
6 5 4
Enter 3 row of numbers (3 numbers, 0 = blank), space-separated:
7 2 8

1 0 3
6 5 4
7 2 8

UCS:
expansions :2
Path: [WRAP_UP]
Moves: 1
Cost: 1.0

A*:
expansions :1
Path: [WRAP_UP]
Moves: 1
Cost: 1.0
giorgosgeorgiades@Giorgoss-MacBook-Air askisi1_2 %
```

Το πρώτο πείραμα έγινε για να αποδείξει στοχευμένα τη λειτουργία των κινήσεων wrap. Παρατηρούμε ότι η UCS χρειάστηκε να κάνει μια επιπλέον επέκταση σε σχέση με την A* καθώς πρέπει να εξερευνήσει κάθε πιθανό μονοπάτι μέχρι να φτάσει στον στόχο, ενώ η A* λόγω της ευρετικής μπορεί να επεκταθεί απευθείας προς το φθηνότερο μονοπάτι προς τον στόχο.

```
[giorgosgeorgiades@Giorgoss-MacBook-Air askisi1_2 %
[giorgosgeorgiades@Giorgoss-MacBook-Air askisi1_2 % javac *.java
[giorgosgeorgiades@Giorgoss-MacBook-Air askisi1_2 % java Main
Enter 1 row of numbers (3 numbers, 0 = blank), space-separated:
1 2 0
Enter 2 row of numbers (3 numbers, 0 = blank), space-separated:
6 5 4
Enter 3 row of numbers (3 numbers, 0 = blank), space-separated:
3 7 8

1 2 0
6 5 4
3 7 8

UCS:
expansions :8
Path: [TELEPORT, RIGHT]
Moves: 2
Cost: 1.5

A*:
expansions :2
Path: [TELEPORT, RIGHT]
Moves: 2
Cost: 1.5
giorgosgeorgiades@Giorgoss-MacBook-Air askisi1_2 %
```


Το δεύτερο πείραμα έγινε επίσης για να αποδείξει στοχευμένα την αναγνώριση της teleport κίνησης. Εδώ παρατηρούμε δραματική αύξηση των επεκτάσεων της UCS καθώς χρειαζόμαστε 2 κινήσεις. Στη UCS ο κόμβος της AK θα επεκταθεί προς κάθε γειτονική κατάσταση και μέσω priority queue θα επιλέξει τη φθηνότερη (teleport cost = 0.5), και έπειτα πάλι θα επεκταθεί σε κάθε γειτονική κατάσταση μέχρι να εντοπίσει τον στόχο (TK). Αντίθετα, η A* θα εντοπίσει απευθείας μέσω ευρετικής συνάρτησης τη φθηνότερη κίνηση (teleport) και θα επεκταθεί στον αντίστοιχο κόμβο, και αντίστοιχα θα εντοπίσει και τη TK και θα επεκταθεί σε αυτήν.

```
giorgosgeorgiades@Giorgoss-MacBook-Air askisi1_2 % java Main
Enter 1 row of numbers (3 numbers, 0 = blank), space-separated:
4 3 8
Enter 2 row of numbers (3 numbers, 0 = blank), space-separated:
1 0 6
Enter 3 row of numbers (3 numbers, 0 = blank), space-separated:
5 7 2

4 3 8
1 0 6
5 7 2

UCS:
expansions :307550
Path: [DOWN, LEFT, WRAP_DOWN, DOWN, RIGHT, DOWN, RIGHT, WRAP_DOWN, TELEPORT, UP, WRAP_LEFT, UP, LEFT, WRAP_UP]
Moves: 14
Cost: 13.5

A*:
expansions :123
Path: [RIGHT, UP, TELEPORT, RIGHT, RIGHT, UP, UP, WRAP_RIGHT, DOWN, RIGHT, RIGHT, UP, LEFT, WRAP_UP]
Moves: 14
Cost: 13.5
giorgosgeorgiades@Giorgoss-MacBook-Air askisi1_2 % █
```

```
giorgosgeorgiades@Giorgoss-MacBook-Air askisi1_2 % java Main
Enter 1 row of numbers (3 numbers, 0 = blank), space-separated:
4 5 6
Enter 2 row of numbers (3 numbers, 0 = blank), space-separated:
3 2 1
Enter 3 row of numbers (3 numbers, 0 = blank), space-separated:
0 8 7

4 5 6
3 2 1
0 8 7

UCS:
expansions :246142
Path: [TELEPORT, DOWN, WRAP_RIGHT, UP, WRAP_LEFT, DOWN, WRAP_RIGHT, DOWN, WRAP_LEFT, LEFT, UP, UP, WRAP_UP]
Moves: 13
Cost: 12.5

A*:
expansions :182
Path: [TELEPORT, DOWN, WRAP_RIGHT, UP, WRAP_LEFT, DOWN, WRAP_RIGHT, DOWN, WRAP_LEFT, LEFT, UP, UP, WRAP_UP]
Moves: 13
Cost: 12.5
giorgosgeorgiades@Giorgoss-MacBook-Air askisi1_2 % █
```



```

giorgosgeorgiades@Giorgoss-MacBook-Air askisi1_2 % java Main
Enter 1 row of numbers (3 numbers, 0 = blank), space-separated:
7 0 8
Enter 2 row of numbers (3 numbers, 0 = blank), space-separated:
6 5 4
Enter 3 row of numbers (3 numbers, 0 = blank), space-separated:
1 2 3

7 0 8
6 5 4
1 2 3

UCS:
expansions :11281
Path: [LEFT, WRAP_UP, TELEPORT, LEFT, WRAP_UP, LEFT, TELEPORT, WRAP_UP, LEFT]
Moves: 9
Cost: 8.0

A*:
expansions :51
Path: [LEFT, WRAP_UP, TELEPORT, LEFT, WRAP_UP, LEFT, TELEPORT, WRAP_UP, LEFT]
Moves: 9
Cost: 8.0
giorgosgeorgiades@Giorgoss-MacBook-Air askisi1_2 %

```

Μετα απο τρία πιο σύνθετα πειράματα φτάνουμε αισίως στο συμπέρασμα ότι η A* λόγω ύπαρξης **αποδεκτής** ευρετικής συνάρτησης θα επεκτείνεται σε πολυ λιγότερους κόμβους απο την UCS επειδή θα μπορεί να εντοπίζει πρώτα τα φθηνότερα μονοπάτια και μετα να επεκτείνεται πάνω σε αυτά, ενώ η UCS θα πρέπει να επεκτείνεται πρώτα εξαντλητικά προς κάθε διαθέσιμο μονοπάτι και μετά λόγω σύγκρισης κόστους να επιλέγει το φθηνότερο.